

Infinite loop

In computer programming, an **infinite loop** (or **endless loop**)^{[1][2]} is a sequence of instructions that, as written, will continue endlessly, unless an external intervention occurs, such as turning off power via a switch or pulling a plug. It may be intentional.

There is no general algorithm to determine whether a computer program contains an infinite loop or not; this is the halting problem.

Overview

An *infinite loop* is a sequence of instructions in a computer program which loops endlessly, either due to the loop having no terminating condition,^[3] having one that can never be met, or one that causes the loop to start over. In older operating systems with cooperative multitasking,^[4] infinite loops normally caused the entire system to become unresponsive. With the now-prevalent preemptive multitasking model, infinite loops usually cause the program to consume all available processor time, but can usually be terminated by a user. Busy wait loops are also sometimes called "infinite loops". Infinite loops are one possible cause for a computer hanging or freezing; others include thrashing, deadlock, and access violations.

Intended vs unintended looping

Looping is repeating a set of instructions until a specific condition is met. An infinite loop occurs when the condition will never be met due to some inherent characteristic of the loop.

Intentional looping

There are a few situations when this is desired behavior. For example, the games on cartridge-based game consoles typically have no exit condition in their main loop, as there is no operating system for the program to exit to; the loop runs until the console is powered off.

Modern interactive computers require that the computer constantly be monitoring for user input or device activity, so at some fundamental level there is an infinite processing idle loop that must continue until the device is turned off or reset. In the Apollo Guidance Computer, for example, this outer loop was contained in the Exec program,^[5] and if the computer had absolutely no other work to do, it would loop run a dummy job that would simply turn off the "computer activity" indicator light.

Multi-threading

In multi-threaded programs some threads can be executing inside infinite loops without causing the entire program to be stuck in an infinite loop. If the main thread exits, all threads of the process are forcefully stopped, thus all execution ends and the process/program terminates. The threads inside the infinite loops

can perform "housekeeping" tasks or they can be in a blocked state waiting for input (from socket/queue) and resume execution every time input is received.

Unintentional looping

Most often, the term is used for those situations when this is not the intended result; that is, when this is a bug.^[6] Such errors are most common by novice programmers, but can be made by experienced programmers also, because their causes can be quite subtle.

One common cause, for example, is that a programmer intends to iterate over sequence of nodes in a data structure such as a linked list or tree, executing the loop code once for each node. Improperly formed links can create a *reference loop* in the data structure, where one node links to another that occurs earlier in the sequence. This makes part of the data structure into a ring, causing naive code to loop forever.



A blue screen of death on Windows XP.
"The device driver got stuck in an infinite loop."

While most infinite loops can be found by close inspection of the code, there is no general method to determine whether a given program will ever halt or will run forever; this is the undecidability of the halting problem.^[7]

Interruption

As long as the system is responsive, infinite loops can often be interrupted by sending a signal to the process (such as SIGINT in Unix), or an interrupt to the processor, causing the current process to be aborted. This can be done in a task manager, in a terminal with the Control-C command,^[8] or by using the kill command or system call. However, this does not always work, as the process may not be responding to signals or the processor may be in an uninterruptible state, such as in the Cyrix coma bug (caused by overlapping uninterruptible instructions in an instruction pipeline). In some cases other signals such as SIGKILL can work, as they do not require the process to be responsive, while in other cases the loop cannot be terminated short of system shutdown.

Language support

Infinite loops can be implemented using various control flow constructs. Most commonly, in unstructured programming this is jump back up (goto), while in structured programming this is an indefinite loop (while loop) set to never end, either by omitting the condition or explicitly setting it to true, as while (true)

Some languages have special constructs for infinite loops, typically by omitting the condition from an indefinite loop. Examples include Ada (loop ... end loop),^[9] Fortran (DO ... END DO), Go (for { ... }), Ruby (loop do ... end), and Rust (loop { ... }).

Infinite loops can also be created using recursion. This process is called infinite recursion.

Examples of intentional infinite loops

This is a loop that will print "Infinite loop" without halting, in C:

```
#include <stdio.h>

int main() {
    for (;;) {
        printf("Infinite loop\n");
    }
    return 0;
}
```

The form `for (;;)` for an infinite loop is traditional, appearing in the standard reference *The C Programming Language*, and is often pronounced "forever."^[10] However, its presence does not guarantee an infinite loop since the loop can be terminated by breaking using `break`, returning using `return`, or jumping using `goto`.

In C and C-derived languages, an infinite loop is perhaps more modernly expressed as `while (true)`:

```
#include <stdio.h>

int main() {
    while (true) {
        printf("Infinite loop\n");
    }
    return 0;
}
```

In the descendant languages of C, such as C++, Java, and C#, it is more idiomatic to use `while (true)`.

A similar example in 1980s-era BASIC:

```
10 PRINT "INFINITE LOOP"
20 GOTO 10
```

A similar example in MS-DOS compatible batch files:

```
:A
echo Infinite Loop
goto :A
```

The while loop never terminates because its condition is always true.

In Bourne Again Shell:

```
for ((;;)); do
    echo "Infinite Loop"
done
```

Examples of unintentional infinite loops

Mathematical errors

Here is one example of an infinite loop in Visual Basic:

```
dim x as integer
do while x < 5
    x = 1
    x = x + 1
loop
```

This creates a situation where x will never be greater than 5, since at the start of the loop code, x is assigned the value of 1 (regardless of any previous value) before it is changed to $x + 1$. Thus the loop will always result in $x = 2$ and will never break. This could be fixed by moving the $x = 1$ instruction outside the loop so that its initial value is set only once.

In some languages, programmer confusion about mathematical symbols may lead to an unintentional infinite loop. For example, here is a snippet in C:

```
#include <stdio.h>

int main(void) {
    int a = 0;
    while (a < 10) {
        printf("%d\n", a);
        if (a = 5) {
            printf("a equals 5!\n");
        }
        ++a;
    }
    return 0;
}
```

The expected output is the numbers 0 through 9, with an interjected "a equals 5!" between 5 and 6. However, in the line "if (a = 5)" above, the = (assignment) operator was confused with the == (equality test) operator. Instead, this will assign the value of 5 to a at this point in the program. Thus, a will never be able to advance to 10, and this loop cannot terminate.

Rounding errors

Unexpected behavior in evaluating the terminating condition can also cause this problem. Here is an example in C:

```
float x = 0.1;
while (x != 1.1) {
    printf("x = %22.20f\n", x);
    x += 0.1;
}
```

C output on an AMD Turion processor:

```
x = 0.10000000149011611938
x = 0.20000000298023223877
x = 0.30000001192092895508
x = 0.40000000596046447754
x = 0.50000000000000000000
x = 0.60000002384185791016
x = 0.70000004768371582031
x = 0.80000007152557373047
x = 0.90000009536743164062
```

On some systems, this loop will execute ten times as expected, but on other systems it will never terminate. The problem is that the loop terminating condition ($x \neq 1.1$) tests for exact equality of two floating point values, and the way floating point values are represented in many computers will make this test fail, because they cannot represent the value 0.1 exactly, thus introducing rounding errors on each increment (cf. box).

```
x = 1.00000011920928955078
x = 1.10000014305114746094
x = 1.20000016689300537109
...
```

The same can happen in Python:

```
x: float = 0.1
while x != 1:
    print(x)
    x += 0.1
```

Because of the likelihood of tests for equality or not-equality failing unexpectedly, it is safer to use greater-than or less-than tests when dealing with floating-point values. For example, instead of testing whether x equals 1.1, one might test whether ($x \leq 1.0$), or ($x < 1.1$), either of which would be certain to exit after a finite number of iterations. Another way to fix this particular example would be to use an integer as a loop index, counting the number of iterations that have been performed.

A similar problem occurs frequently in numerical analysis: in order to compute a certain result, an iteration is intended to be carried out until the error is smaller than a chosen tolerance. However, because of rounding errors during the iteration, the specified tolerance can never be reached, resulting in an infinite loop.

Multi-party loops

An infinite loop may be caused by several entities interacting. Consider a server that always replies with an error message if it does not understand the request. Even if there is no possibility for an infinite loop within the server itself, a system comprising two of them (A and B) may loop endlessly: if A receives a message of unknown type from B , then A replies with an error message to B ; if B does not understand the error message, it replies to A with its own error message; if A does not understand the error message from B , it sends yet another error message, and so on.

One common example of such situation is an email loop. An example of an email loop is if someone receives mail from a no reply inbox, but their auto-response is on. They will reply to the no reply inbox, triggering the "this is a no reply inbox" response. This will be sent to the user, who then sends an auto reply to the no-reply inbox, and so on and so forth.

Pseudo-infinite loops

A pseudo-infinite loop is a loop that appears infinite but is really just a very long loop.

Impossible termination condition

An example for loop in C:

```
for (unsigned int i = 1; i != 0; i++) {}
```

It appears that this will go on indefinitely, but in fact the value of `i` will eventually reach the maximum value storable in an `unsigned int` and adding 1 to that number will wrap-around causing an integer overflow, breaking the loop. The actual limit of `i` depends on the details of the system and compiler used. With arbitrary-precision arithmetic, this loop would continue until the computer's memory could no longer hold `i`. If `i` was a signed integer, rather than an unsigned integer, overflow would be undefined.

See also



- Cycle detection
- Divergence (computer science)
- Infinite regress

References

1. "Endless loop dictionary definition" (<https://www.yourdictionary.com/endless-loop>). Archived (<https://web.archive.org/web/20200801213748/https://www.yourdictionary.com/endless-loop>) from the original on 2020-08-01. Retrieved 2020-01-22.
2. "What is infinite loop (endless loop)" (<https://whatis.techtarget.com/definition/infinite-loop-endless-loop>). Archived (<https://web.archive.org/web/20190715101446/https://whatis.techtarget.com/definition/infinite-loop-endless-loop>) from the original on 2019-07-15. Retrieved 2020-01-22.
3. "Codes and Modes: The Character of Documentary Culture" (<http://www.flowjournal.org/tag/loop-media/>). *Flow Journal*. November 2014. Archived (<https://web.archive.org/web/20200801213624/http://www.flowjournal.org/tag/loop-media/?print=print-search>) from the original on 2020-08-01. Retrieved 2020-01-23. "an infinite loop is one that lacks .. an exit condition"
4. also known as non-preemptive-multitasking: "Non-preemptive Multitasking" (<https://www.pcmag.com/encyclopedia/term/48051/non-preemptive-multitasking>). *PC Magazine*. Archived (<https://web.archive.org/web/20190726232111/https://www.pcmag.com/encyclopedia/term/48051/non-preemptive-multitasking>) from the original on July 26, 2019. Retrieved February 7, 2024.
5. David Hoag (September 1976). "The History of Apollo On-board Guidance, Navigation, and Control" (http://klabs.org/history/history_docs/mit_docs/1711.pdf) (PDF). Charles Stark Draper Laboratory. Archived (https://web.archive.org/web/20161105060425/http://klabs.org/history/history_docs/mit_docs/1711.pdf) (PDF) from the original on 2016-11-05. Retrieved 2020-01-23.
6. "New York Times Crossword Answers" (<https://nyxcrossword.com/2013/10/1013-13-new-york-times-crossword.html>). October 13, 2013. Archived (<https://web.archive.org/web/20200802040416/https://nyxcrossword.com/2013/10/1013-13-new-york-times-crossword.html>) from the original on August 2, 2020. Retrieved January 22, 2020. "computing .. a defect .. which .. to loop"

7. "Halting Problem in Theory of Computation" (<https://www.geeksforgeeks.org/halting-problem-in-theory-of-computation>). 3 October 2018. Archived (<https://web.archive.org/web/20200809100104/https://www.geeksforgeeks.org/halting-problem-in-theory-of-computation/>) from the original on 9 August 2020. Retrieved 22 January 2020.
8. "A Buffer Overflow Exploit Against the DameWare Remote Control software" (<https://pen-testing.sans.org/resources/papers/gcih/buffer-overflow-exploit-dameware-remote-control-software-104168>). December 19, 2003. Archived (<https://web.archive.org/web/20200724200739/https://pen-testing.sans.org/resources/papers/gcih/buffer-overflow-exploit-dameware-remote-control-software-104168>) from the original on July 24, 2020. Retrieved January 22, 2020. "As soon as the command shell is closed with a control-c combination ..."
9. Ada Programming: Control: Endless Loop
10. "Endless loop in C/C++" (<https://stackoverflow.com/questions/20186809/endless-loop-in-c-c>). Archived (<https://web.archive.org/web/20160803202212/http://stackoverflow.com/questions/20186809/endless-loop-in-c-c>) from the original on 2016-08-03.

External links

- Make an infinite loop (<http://www.programming-idioms.org/idiom/50/make-an-infinite-loop>) in several languages, on programming-idioms.org (<http://www.programming-idioms.org/>).
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Infinite_loop&oldid=1330772781"